

Cooperative Coverage Mission Planning for Multi-UAV Based on the Dual-Ring Dynamic Scheduler

Yongzhuo Yu¹, Xuting Duan¹, Feiyang Zhao¹, Jianshan Zhou¹,
Chunmian Lin¹, *Member, IEEE*, Kaige Qu¹, *Member, IEEE*, and Daxin Tian¹, *Fellow, IEEE*

Abstract—Unmanned aerial vehicles (UAVs) have rapidly advanced in applications, such as disaster response, infrastructure inspection, and smart city systems. However, cooperative coverage mission planning in dynamic environments poses a persistent challenge in balancing global optimization with real-time responsiveness. To address this, we propose a two-stage task allocation framework based on a dual-ring dynamic scheduler. In the centralized planning phase, an enhanced NSGA-II algorithm is developed, incorporating dual-population initialization, path-exchange crossover, and multi-strategy mutation. Experimental results demonstrate a 49% improvement in the hypervolume indicator over the baseline NSGA-II, with average reductions of 13.33% and 23.71% in total and maximum execution times, respectively. In the dynamic scheduling phase, we design a distributed auction mechanism leveraging a dual-ring communication topology, capable of handling six types of events: UAV failure, UAV addition, node cancel exploration, node urgent exploration, node re-exploration and node in-depth exploration. Through event-driven auctions and group-based bidding, the system maintains a load imbalance under 28% and achieves effective rebalancing even under scenarios with over 50% UAV loss. These results validate the robustness and adaptability of the dual-ring dynamic scheduler in real-time collaborative coverage missions. The proposed method demonstrates significant potential in dynamic, large-scale UAV applications. Source code will be available at: https://github.com/GradualScholar/Coverage_Mission_Planning.

Index Terms—Coverage mission planning, distributed, dynamic collaboration, multi-unmanned aerial vehicle (UAV), UAV swarm coordination.

Received 30 June 2025; revised 18 July 2025; accepted 12 August 2025. Date of publication 18 August 2025; date of current version 24 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62173012, Grant 62432002, Grant U2433202, and Grant U22A2046; and in part by the Fundamental Research Funds for the Central Universities (Beihang Ganwei Action Plan Key Program) under Grant JK2024-19. (*Corresponding author: Xuting Duan.*)

Yongzhuo Yu, Feiyang Zhao, Jianshan Zhou, Chunmian Lin, and Kaige Qu are with the State Key Laboratory of Intelligent Transportation System, and the Beijing Key Laboratory for Cooperative Vehicle Infrastructure Systems and Safety Control, School of Transportation Science and Engineering, Beihang University, Beijing 100191, China (e-mail: yuyongzhuo@buaa.edu.cn; sy2413315@buaa.edu.cn; jianshanzhou@foxmail.com; cmlin@buaa.edu.cn; kaigequ@buaa.edu.cn).

Xuting Duan and Daxin Tian are with the State Key Laboratory of Intelligent Transportation System, and the Beijing Key Laboratory for Cooperative Vehicle Infrastructure Systems and Safety Control, School of Transportation Science and Engineering, Beihang University, Beijing 100191, China, and also with Zhongguancun Laboratory, Beijing 100081, China (e-mail: duanxuting@buaa.edu.cn; dtian@buaa.edu.cn).

Digital Object Identifier 10.1109/IJOT.2025.3599156

I. INTRODUCTION

THE RAPID development of the low-altitude economy has driven continuous advancements in the technology of unmanned aerial vehicle (UAV) systems, pushing it toward higher levels of automation and intelligence [1]. UAVs have been widely applied in various fields, including agricultural management (precision spraying, seeding) [2], logistics delivery [3], environmental monitoring [4], disaster rescue [5], facility inspection [6], and urban surveillance [7]. Compared to the intelligence of individual UAVs, small UAV swarms have demonstrated superior performance in both military and civilian scenarios. However, increasing the number of UAVs in a swarm leads to an exponential growth in computational and planning complexity [8], and improving the level of swarm intelligence remains a critical issue that needs to be addressed.

Compared to the intelligence of individual UAVs, small UAV swarms have demonstrated superior performance in both military and civilian scenarios. However, increasing the number of UAVs in a swarm leads to an exponential growth in computational and planning complexity [8], and improving the level of swarm intelligence remains a critical issue that needs to be addressed. Multi-UAV aerial operations offer the capability to rapidly cover large-scale areas, significantly improving operational efficiency [9]. As highlighted by Zhichkin et al. [10] agricultural plant protection UAVs exhibit the highest economic effectiveness, achieving more than double the efficiency of conventional self-propelled sprayers. However, the complexity of multi-UAV coverage missions escalates substantially with increasing task scale and environmental uncertainty [11]. In dynamic environments, achieving a balance between global optimization and real-time responsiveness becomes a core challenge, particularly as the need for effective communication among UAVs grows [12]. In response to the increasing complexity of multiagent systems, communication topologies, such as single-ring and star structures are commonly employed due to their architectural simplicity and ease of implementation [13]. In a single-ring topology, agents are connected in a circular configuration, enabling message passing along the ring. While this structure is effective in small-scale deployments, it encounters significant limitations in scalability, particularly in terms of node management and fault tolerance as the number of agents increases [14]. Similarly, the star topology, which relies on a central coordinating node, introduces a single point of failure that can lead to a complete system breakdown if the central

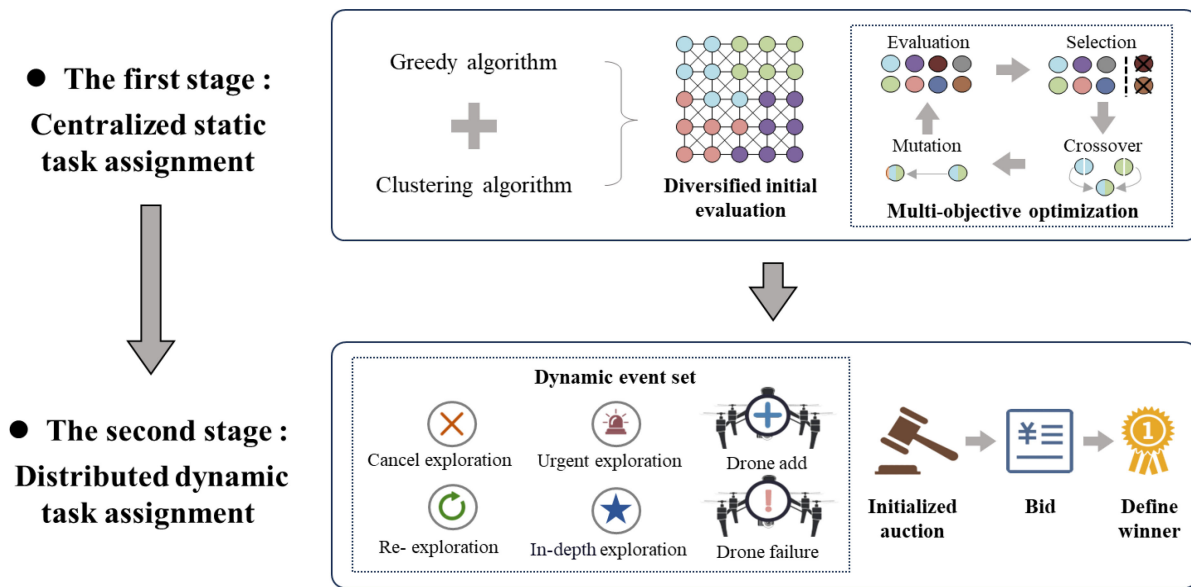


Fig. 1. Two-stage task allocation framework: centralized static task allocation and distributed dynamic task allocation.

node malfunctions. These inherent drawbacks underscore the need for more resilient and scalable communication strategies. As a result, recent research has increasingly focused on the development and adoption of alternative topologies that offer enhanced fault tolerance and adaptability for complex multi-UAV systems [15].

Building upon the need for robust and scalable communication topologies, another critical aspect in multi-UAV coordination lies in the balance between centralized and distributed control paradigms. Existing research on multi-UAV coordination predominantly focuses on either centralized static allocation or distributed dynamic scheduling [16]. Centralized approaches, while capable of achieving globally optimal task assignments, often suffer from poor scalability and limited responsiveness in dynamic or uncertain environments, where task demands may change rapidly and unpredictably [17]. Conversely, distributed dynamic scheduling improves adaptability and fault tolerance by enabling agents to make decisions locally; however, it frequently lacks a global perspective, leading to potential drawbacks, such as task duplication, suboptimal resource usage, and inefficient coverage [18]. These inherent tradeoffs highlight a critical gap in current methodologies—namely, the absence of a unified framework that effectively integrates the global awareness of static planning with the real-time responsiveness of dynamic scheduling, while maintaining scalability and resilience in complex mission scenarios.

To address these challenges, we propose a cooperative coverage mission planning framework for multi-UAV systems based on a dual-ring dynamic scheduler. The proposed framework integrates a two-stage task allocation mechanism that combines centralized global optimization with distributed dynamic decision-making. In the static phase, an enhanced NSGA-II algorithm accelerates Pareto front convergence through dual-population initialization and hybrid variation

strategies. In the dynamic phase, a distributed auction mechanism with a dual-ring communication topology is employed to respond effectively to various operational events, ensuring both scalability and robustness. This hybrid approach bridges the gap between static optimization and dynamic adaptability, offering a promising solution for real-time, large-scale UAV coordination in complex and uncertain environments. Building on this framework, this article introduces a two-stage static–dynamic integrated task allocation architecture for unmanned systems, as illustrated in Fig. 1. The core contributions of this work are summarized as follows.

- 1) *Scalable Hybrid Architecture Design*: A two-stage decision-making framework is developed, integrating centralized global optimization with distributed dynamic responsiveness. To reflect real-world complexity, six representative types of dynamic events are modeled, enabling the proposed system to effectively balance global convergence and real-time adaptability in large-scale, uncertain environments.
- 2) *Centralized Static Task Allocation*: In the static phase, an enhanced NSGA-II algorithm is introduced to improve the quality and diversity of the Pareto front. This is achieved through dual-population initialization based on greedy cluster collaboration, path-exchange crossover for local path refinement, and multistrategy mutation operators for maintaining population diversity.
- 3) *Distributed Dynamic Task Allocation*: In the dynamic phase, an adaptive auction-based task allocation algorithm is employed, operating over a dual-ring communication topology. This design improves both scalability and fault tolerance compared to conventional single-ring structures. The ring-based auction mechanism enables efficient task redistribution and workload balancing in response to dynamic changes, while the dual-ring topology supports rapid task reassignment and

system robustness in the event of communication failures or UAV dropouts.

The remainder of this article is organized as follows. Section II reviews related work in the field of coverage-based task allocation. Section III presents a formal problem description, outlining the modeling environment and key assumptions. Section IV details the algorithmic design, including both a centralized static task allocation algorithm and a distributed dynamic counterpart. Section V evaluates the performance of the proposed algorithms through a series of experiments and provides a comprehensive analysis of the results. Finally, Section VI concludes this article and outlines potential directions for future research.

II. LITERATURE REVIEW

A. Coverage Path Planning for a Single UAV

Coverage path planning (CPP) is an important subfield of robot motion planning, with the core objective of constructing an optimal motion trajectory that fully traverses the target area for mobile robots. This problem can be further decoupled into two core subtasks: cell decomposition and coverage motion planning [19].

In the cell decomposition phase, the target area is discretized into nonoverlapping basic units (Elementary Cells). The unit scale typically aligns with the agent's sensing radius and coverage mode (ground/air) [20]. In theory, unit resolution is negatively correlated with planning efficiency: larger units may require secondary decomposition or multiple rounds of coverage, while high-resolution units can be covered in a single pass but significantly increase computational complexity [21]. As shown in Fig. 2, existing decomposition methods can be categorized into three types.

- 1) Geometric-topological decomposition methods, including 3-D convex decomposition based on nonuniform rational B-splines (NURBS) [22] and Boustrophedon concave-convex decomposition based on critical point detection [23].
- 2) Unit approximation methods, such as regular grid (square/hexagonal) discretization [24] and Delaunay triangulation [25].
- 3) Hybrid decomposition strategies, which integrate topological preservation and computational efficiency advantages through graph structure transformation methods [26].

By mapping continuous space to a topologically equivalent discrete graph model (e.g., Reeb graph [27]), these methods can effectively reduce path search complexity. It is worth noting that each decomposition paradigm involves significant tradeoffs between computational complexity, environmental adaptability, and path optimality, requiring careful selection of the paradigm based on the specific application scenario [28].

In coverage motion planning, it is essential to systematically integrate multidimensional constraints, such as the agent's kinematic constraints, sensor perception characteristics, and dynamic obstacle avoidance, to construct feasible paths that meet practical engineering requirements. In airspace coverage scenarios, workspace obstacles are typically modeled as no-fly

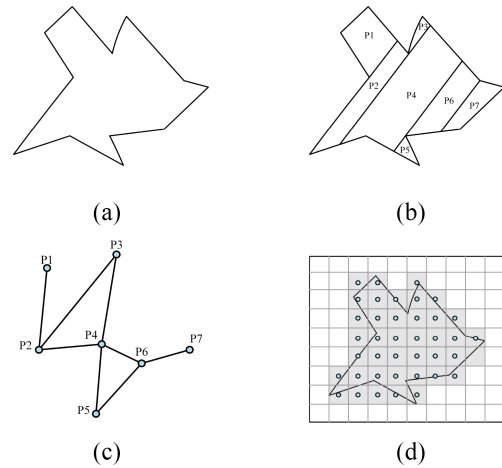


Fig. 2. Cell decomposition method. (a) Region of interest. (b) Geometric topological decomposition. (c) Mixed decomposition. (d) Element approximation.

zones (NFZs) [29], and their spatial representation must adhere to international airspace control standards (e.g., the ICAO DOC 4444 standard). Typical application scenarios include the 3-D exclusion of sensitive areas, such as airport core zones and nuclear facility protection areas [30]. Current path planning methodologies are primarily based on graph search algorithms [31] (e.g., A* algorithm [32], Dijkstra's algorithm [33], JPS algorithm), sampling-based algorithms (e.g., RRT [34], PRM, and subsequent improvements to RRT-based algorithms), and intelligent algorithms (e.g., genetic algorithms, ant colony optimization, particle swarm optimization, etc.) [35], [36], [37]. Coverage exploration planning for individual intelligence has a solid research foundation and is gradually evolving toward swarm intelligence [38].

B. Collaborative Mission Planning in Multi-UAV Systems

Unlike traditional individual intelligence, swarm intelligence in unmanned aerial systems requires global optimization at the system level to achieve task allocation [39], resource coordination, and behavior synchronization. The core challenge lies in balancing individual autonomy and cooperation through task planning mechanisms [40]. The efficiency of swarm cooperation heavily depends on the rationality of task decomposition and allocation, requiring multiobjective optimization models that simultaneously satisfy constraints, such as coverage, energy balance, and real-time responsiveness [41]. Existing cooperative architectures can be primarily categorized into centralized and distributed approaches [42]. Centralized architectures (e.g., improved NSGA-II [43], mixed-integer linear programming [44]) rely on global information modeling to generate Pareto optimal solutions, offering theoretical advantages in static or low-dynamic environments. However, their high computational complexity and strong dependence on communication limit their real-time applicability. In contrast, distributed architectures (e.g., auction algorithms [45], self-organizing network protocols [46]) achieve rapid response through local negotiation, demonstrating strong robustness in scenarios involving UAV failures or

sudden tasks. However, they are prone to getting stuck in local suboptimal solutions [47].

To balance global optimization and dynamic adaptability, hybrid architectures commonly adopt mechanisms, such as “offline global optimization–online local correction” (e.g., the STA-DTA combined framework proposed by Kang et al. [48]) or “central-edge” coordination models [49]. Despite their conceptual advantages, the effectiveness of these hybrid approaches is often constrained by the underlying communication topologies. Previous studies have predominantly employed single-ring and star topologies in multiagent systems due to their structural simplicity and minimal communication overhead. Nonetheless, these topologies exhibit significant limitations in terms of scalability and fault tolerance [50]. For example, the single-ring topology lacks redundancy, making it inefficient for large-scale systems—any single link failure can potentially disrupt communication across the entire network. Similarly, the star topology, although effective in small-scale scenarios, introduces a critical single point of failure at the central node, thereby compromising system robustness in the event of node malfunction or overload. Building on these observations, recent advances in multiagent task allocation have explored optimization-based swarm intelligence algorithms (e.g., MILP models [51]) and market mechanism-based dynamic protocols [45]. The former is well-suited for addressing multiconstraint combinatorial optimization problems but lacks the flexibility to adapt in real time. In contrast, the latter enhances distributed decision-making through auction-based strategies, yet often fails to guarantee global convergence.

In dynamic task allocation research, UAV swarms face challenging conditions, such as communication constraints, highly dynamic environments [52], and resource heterogeneity, making distributed architectures a core focus for technological breakthroughs [17]. Current cutting-edge methods focus on the design of decentralized autonomous coordination mechanisms, such as distributed reinforcement learning (DRL) [53], [54] and topology-aware fusion frameworks [55]. For instance, Wang et al. adopted a multitask deep reinforcement learning algorithm that improves sample efficiency and reward performance by utilizing shared information between tasks in the environment. They used a dense reward system to facilitate efficient UAV training, successfully guiding UAV missions [56]. However, while multiagent deep reinforcement learning (MADRL) optimizes decision-making through end-to-end training, its dependence on global state information and high sample complexity presents deployment challenges in harsh environments, such as strong battlefield interference and high dynamics [57].

In summary, CPP and UAV swarm coordination are core research directions in the field of robotic intelligence. Their theoretical development and technological breakthroughs consistently focus on achieving a dynamic balance between environmental representation accuracy, planning algorithm efficiency, and system robustness. Existing research on unit decomposition paradigms has led to three main methodological frameworks: geometric topology, cell approximation, and hybrid strategies. However, each of these approaches still faces significant limitations in terms of dynamic environment

adaptability and the handling of multiconstraint coupling. In the field of cooperative task allocation, centralized architectures ensure global optimality but are constrained by high computational complexity, while distributed strategies enhance real-time performance but are prone to getting stuck in local suboptimal solutions. Both approaches face theoretical and technical challenges in dynamic event handling and heterogeneous resource scheduling.

III. PROBLEM DESCRIPTION

In the task allocation for multi-UAV coverage, the goal is to balance the workload of each UAV while ensuring that the assigned tasks are well-connected. This improves the task completion efficiency of each UAV and reduces interference between UAVs. After task allocation, the UAVs autonomously plan their coverage paths to achieve efficient coverage.

Under the premise of ensuring the validity of the model, reasonable assumptions are made for the proposed model to reduce the difficulty of solving the model: the UAV has enough resources to perform the task, and the UAV is abstracted as a heptad $UAV_i = (s_i, \text{pos}_i, v, c_i, q_i, p_i, x_i)$. s_i denotes the state of the UAV at the point in time, the two attributes of active and faulty as shown in

$$s_i = \begin{cases} 0, & \text{active} \\ 1, & \text{failure.} \end{cases} \quad (1)$$

$\text{pos}_i = (x_i, y_i)$ indicates the position of UAV_q at time t . v denotes the flight speed of the UAV. c_i indicates the current mission of the UAV at time t . q_i indicates the current queue of pending tasks at time t . x_i represents the set of tasks performed by UAV i . Mission set $M = \{m_1, m_2, \dots, m_K\}$ indicates the missions to be performed in the region of interest. To better describe the properties of the task, it is represented by the triad $m_K = \{s'_i, \text{pos}', a\}$. Included among these, s'_i indicates the statement of mission m_K at time t : Located in the current task, in the pending queue, in the set of completed tasks. x_i represents the node list of UAV_i as shown in (2). $\text{pos}' = (x, y)$ indicates the location of the task. a indicates the time required to complete the task m_K . This time is constant, but the task completion time can change due to dynamic events

$$x_i = \{m_K | m_K \in M \wedge x_{ik} = 1\}. \quad (2)$$

In exploration missions, the task assignment problem for UAV swarms can be formulated as a balanced connected q -partition (BCPq) problem. Notably, since any feasible q -partition of the network nodes inherently satisfies connectivity constraints, the computational complexity of brute-force search escalates exponentially to $O(q^n)$ thereby establishing the NP-hard nature of this combinatorial optimization challenge. As shown in Fig. 3 Given a topology graph $G = (V, E, W)$, where V is the set of vertices, E is the set of edges, and W is the weight function for the vertices, with the weight representing the time required to cover a specific area, the objective is to find an optimal partition $Q = (V_1, V_2, \dots, V_q)$ such that each task is connected and the search time for each subtask is minimized, as shown in the following:

$$\min_Q \max_i \frac{w(V_i)}{\text{vel}_i}, \text{ for all feasible } Q. i \in \{1, 2, \dots, q\}. \quad (3)$$

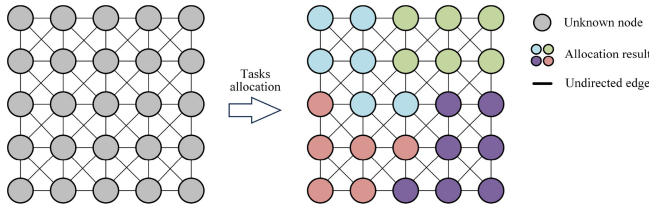


Fig. 3. Illustration of multi-UAV coverage task allocation.

Here, q represents the number of UAVs, $w(V_i) = K$ denotes the number of nodes in the partition V_i , and vel_i refers to the speed of the UAV. Let the UAV's sensing range be d . The specific completion time for UAV j to perform subtask i is given by $w_i/d/vel_j$.

The optimization objective during the task allocation phase aims to achieve dual goals: first, to minimize the total completion time of the overall coverage elitist task, which corresponds to solving the minimization objective function $\min t_a$

$$t_a = \frac{d_e}{v} + \sum m_k(a), m_k \in x_n \quad (4)$$

where, d_e represents the total distance traveled by the UAV during task execution, while $\sum m_k(a)$ denotes the time expenditure for the UAV to complete the task.

The second goal is to minimize the maximum execution time, i.e., to solve the minimization objective $\min t$, where t represents the time consumed by the longest task sequence in the multitask sequence. Specifically, t is equal to the maximum value of the UAV's task completion time set, expressed as

$$\max \left(\frac{\sum_{j=1}^n x_{ji} w_j}{vel_i} \right) \forall i \in \{1, 2, \dots, q\}. \quad (5)$$

In particular, the overall coverage task completion time reflects the total time from the initiation to the completion of all tasks, while the maximum execution time focuses on the longest individual task sequence in the task set, serving as another key indicator for evaluating task execution efficiency. The mathematical model is constructed as shown in (6)–(9) above

$$\min C = \min (t_a, t) \quad (6)$$

$$\sum_{i=1}^q x_{ik} \geq 1 \quad \forall UAV_q \in UAV \quad (7)$$

$$\sum_{k=1}^K x_{ik} = 1 \quad \forall m_k \in M \quad (8)$$

$$\sum_{i=1}^q \sum_{k=1}^K x_{ik} = K, i = 1, 2, \dots, q; k = 1, 2, \dots, K. \quad (9)$$

Here, q represents the number of UAVs in the system. K represents the total number of task nodes.

IV. ALGORITHM DESIGN

The cooperative dynamic coverage problem addressed in this study is formulated as a multiobjective dynamic multiple traveling salesman problem (MODMTSP), which fundamentally differs from the classical traveling salesman problem (TSP) due to the involvement of multiple agents, conflicting objectives, and real-time dynamic changes. Specifically,

the optimization process seeks to minimize both the total execution time—reflecting overall system efficiency—and the maximum execution time among all UAVs—ensuring balanced workload distribution. These two objectives are inherently conflicting in large-scale systems. Therefore, rather than pursuing a single global optimum, the goal shifts to approximating the Pareto-optimal front that captures tradeoff solutions across multiple criteria.

To solve the multiobjective dynamic multiple TSP (MODMTSP) under large-scale and dynamic conditions, we propose a novel dual-stage hybrid architecture that integrates centralized global planning with decentralized real-time adaptation, as illustrated in Fig. 4. The coverage process is divided into two distinct phases: a static planning phase and a dynamic execution phase. In the static phase, a central server performs global offline task allocation using an enhanced NSGA-II algorithm. This algorithm improves both the diversity and convergence of the Pareto front through three key innovations: 1) a dual-population initialization strategy that combines greedy heuristics with clustering-based solution partitioning; 2) a path-exchange crossover operator that enables partial route exchange between individuals to explore alternative task orderings; and 3) multistrategy mutation mechanism that leverages multiple mutation operators to maintain population diversity and avoid premature convergence. These enhancements allow for effective preoptimization of task sequences, which are then disseminated to all UAVs for synchronized execution.

In the dynamic execution phase, a distributed, event-driven task reallocation mechanism is employed to address six categories of dynamic events, including UAV failures, new UAV additions, and node rediscovery. These events are integrated into an adaptive control loop that governs real-time coordination. To ensure scalable and resilient communication, we introduce a dual-ring communication topology that improves fault tolerance, reduces recovery latency, and enhances network robustness compared to traditional single-ring or broadcast-based models. Within this structure, we further propose a ring-based adaptive auction mechanism, where task reallocation is selectively triggered based on the type and scope of each dynamic event. This localized auction mechanism enables prompt reallocation while preserving global coordination, striking a practical balance between responsiveness and solution consistency. Unlike existing approaches that rely solely on centralized optimization—often lacking adaptability—or purely local heuristics that sacrifice global efficiency, our hybrid framework explicitly integrates multiobjective global planning with decentralized, real-time coordination. It is designed to handle real-time dynamic events, operate under partial information, and maintain system-wide task balance, thereby enhancing both the robustness and scalability of multi-UAV cooperative coverage missions.

A. Phase 1 (Centralized Static Task Allocation)

The improved NSGA-II algorithm is used to solve the multiobjective optimization problem. Through fast non-dominated sorting and the crowding distance comparison mechanism, it efficiently handles conflicting objectives in task allocation and generates a uniformly distributed Pareto

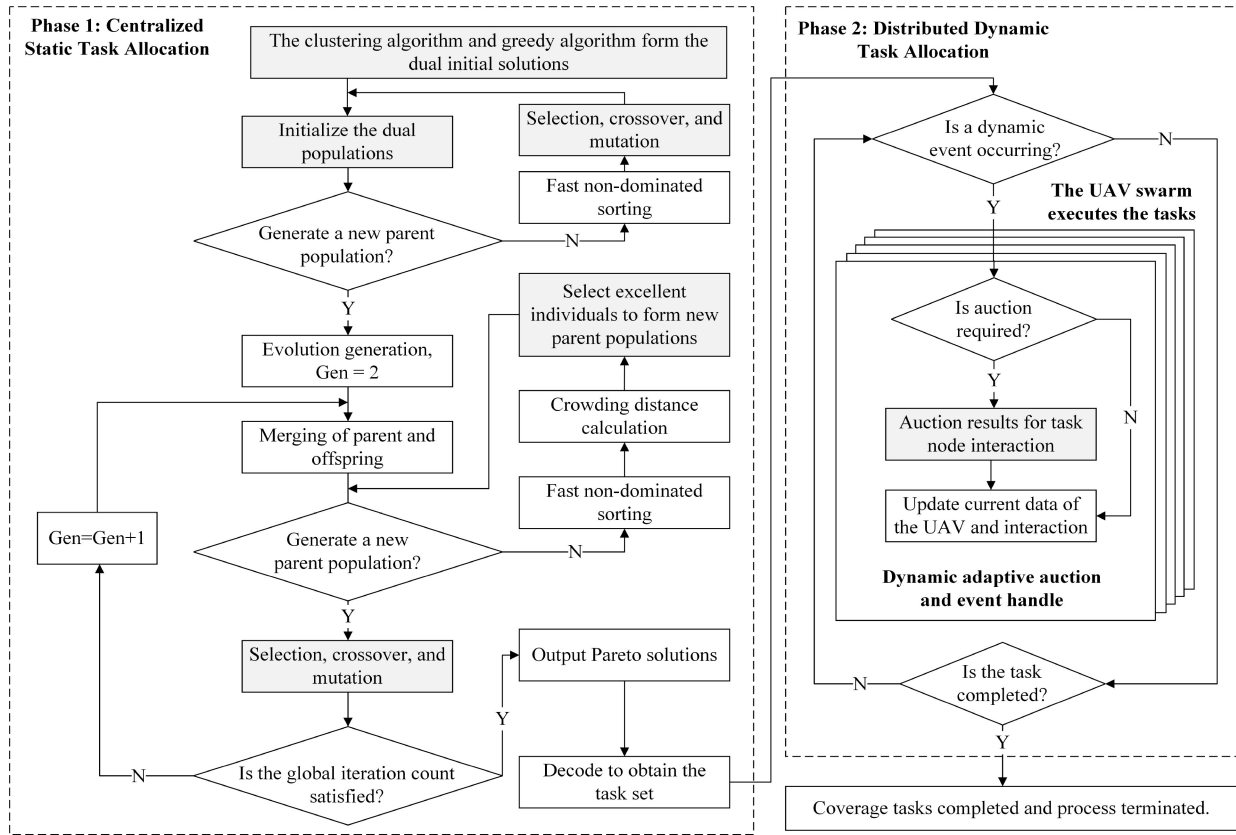


Fig. 4. System flowchart (dark-colored box indicates improvement).

optimal solution set. For the global optimization requirements in static scenarios, three improvements are made to the NSGA-II algorithm in this article: initial dual-population construction, path exchange crossover, and a hybrid mutation strategy. Additionally, an elitist retention strategy and tournament selection are employed to ensure the quality of the population.

- 1) *Initial Dual-Population Construction*: To enhance both the quality and diversity of initial solutions, we adopt a dual-population initialization strategy that integrates a greedy heuristic with a clustering-based task grouping method. This hybrid approach aims to generate two structurally distinct populations, thereby promoting a more effective exploration of the solution space from the outset. The first population is constructed using a greedy algorithm, which rapidly generates high-quality candidate solutions through local optimization. Tasks are assigned based on minimal-cost metrics, such as Euclidean distance or current UAV load, resulting in efficient yet structurally similar solutions. While effective in producing feasible solutions quickly, this method alone tends to limit population diversity due to convergence around similar task assignments. To counteract this limitation, the second population is initialized using a spatial clustering technique—such as *K*-means—which partitions the task set into geographically coherent clusters. Each cluster is then assigned to a UAV, forming distinct task allocation patterns that differ significantly from those produced by the greedy

approach [58]. This clustering-based initialization introduces structural heterogeneity, thereby enhancing the diversity of coordination strategies represented in the population. By combining these two complementary strategies, the dual-population framework ensures that the initial population is not only feasible and high-performing, but also spans a broader and more diverse region of the search space. This diversity is critical for avoiding premature convergence and enables the algorithm to more robustly explore the tradeoff surface of the Pareto front. The complete initialization process is summarized in Algorithm 1. First, the greedy algorithm generates q_1 task sequences. Each sequence x_n satisfies both distance and load balancing constraints, as shown in

$$x_i = \arg \min_x (\alpha \cdot D(x) + (1 - \alpha) \cdot \text{Var}(L(x))). \quad (10)$$

Here, $D(x)$ represents the total distance of the task sequence, $\text{Var}(L(x))$ denotes the load variance, and α is the weight coefficient. The solution space through *K*-means clustering. For each cluster C_q , a task sequence x_k is generated such that the shortest path within the cluster is minimized, as shown in the following:

$$x_k = \arg \min_x \sum_{v_i, v_j \in x} d(v_i, v_j). \quad (11)$$

Here, $d(v_i, v_j)$ represents the shortest path distance between nodes v_i and v_j .

Algorithm 1 Initial Dual-Population Construction

Input: Task node set V , UAV set U , population size N , greedy population size N_1 , clustering population size N_2 , number of clusters q

Output: Initial population P_0

```

1: //Generate greedy population
2:  $P_{\text{greedy}} \leftarrow \emptyset$ 
3: for  $i = 1$  to  $N_1$  do
4:    $\pi_i \leftarrow \text{GreedyTaskAllocation}(V, U)$ 
5:    $P_{\text{greedy}} \leftarrow P_{\text{greedy}} \cup \{\pi_i\}$ 
6: end for
7: //Generate clustering population
8:  $P_{\text{cluster}} \leftarrow \emptyset$ 
9:  $C, A \leftarrow \text{ShortestPathKMeans}(V, U, q)$ 
10: for each  $C_q \in C$  do
11:    $x_k \leftarrow \arg \min_{v_i \in \pi} d(v_i, v_j)$ 
12:    $P_{\text{cluster}} \leftarrow P_{\text{cluster}} \cup \{x_k\}$ 
13: end for
14: //Merge populations
15:  $P_0 \leftarrow P_{\text{greedy}} \cup P_{\text{cluster}}$ 
16: while  $|P_0| < N$  do
17:    $x_{\text{exchange}} \leftarrow \text{Repair}(\cdot)$ 
18:    $P_0 \leftarrow P_0 \cup \{x_{\text{exchange}}\}$ 
19: end while
20: return  $P_0$ 

```

Finally, the population is merged and adaptively completed. The greedy population and the clustering population are combined, generating populations N_1 and N_2 based on q_1 and q_2 , respectively, to form the initial population P_0

$$P_0 = \left\{ x_1^{\text{greedy}}, \dots, x_{N_1}^{\text{greedy}} \right\} \cup \left\{ x_1^{\text{cluster}}, \dots, x_{N_2}^{\text{cluster}} \right\}. \quad (12)$$

- 2) *Crossover and Mutation Strategies:* To enhance the inheritance of feasible paths in offspring and maintain population diversity, thereby broadening the search depth, this article proposes a distance-constrained path exchange crossover and hybrid mutation strategy. Let the parent individuals be $\pi_1 = (v_{a_1}, v_{a_2}, \dots, v_{a_n})$ and $\pi_2 = (v_{b_1}, v_{b_2}, \dots, v_{b_n})$, and the node distance matrix be $D = [d_{ij}]_{n \times n}$. The parent path exchange point selection rule is as follows, (13), where nodes that are within a predefined distance threshold are exchanged

$$S = \{(i, j) | d_{a_i b_j} \leq \delta\}. \quad (13)$$

Here, S represents the predefined distance threshold, and $d_{a_i b_j}$ denotes the shortest path distance between nodes v_{a_i} and v_{b_j} . To ensure the feasibility and topological validity of task allocation solutions, we implement a dedicated $\text{Repair}(\cdot)$ function that systematically corrects infeasible assignments introduced during population initialization or evolutionary updates. The repair process operates in three stages: first, the function scans the task lists of all UAVs to identify and remove any duplicate task

Algorithm 2 Improved Crossover and Mutation Strategy

Input: Parent individuals π_1 and π_2 , distance matrix D , distance threshold δ , local reordering probability p_{local}

Output: Offspring individuals π'_1 and π'_2 , mutated individual π'

```

1: //Crossover strategy
2:  $\text{SwapPoints} \leftarrow \emptyset$ 
3: for  $i = 1$  to  $n$  do
4:   for  $j = 1$  to  $n$  do
5:      $S \leftarrow \{(i, j) | d_{a_i b_j} \leq \delta\}$ 
6:      $\text{SwapPoints} \leftarrow \text{SwapPoints} \cup (i, j)$ 
7:   end for
8: end for
9:  $(i^*, j^*) \leftarrow \text{RandomChoice}(\text{SwapPoints})$ 
10:  $\pi'_1 \leftarrow (v_1^1, \dots, v_{i^*}^1, v_{j^*+1}^2, \dots, v_n^2), \pi'_2 \leftarrow (v_1^2, \dots, v_{j^*}^2, v_{i^*+1}^1, \dots, v_n^1)$ 
11: //Mutation strategy
12: if  $\text{Random}(0, 1) < p_{\text{local}}$  then
13:    $\text{RandomChoice}(v_i, v_{i+1}, \dots, v_j)$ 
14:    $\pi'_{\text{sub}} = \arg \min_{\pi_{\text{sub}}} \sum_{k=i}^{j-1} d_{k, k+1}$ 
15: else
16:    $\text{RandomChoice}(v_i, v_j)$ 
17:    $\text{Compute } f(\pi')$ 
18: end if
19: if  $f(\pi') < f(\pi)$  then
20:   exchange  $v_i, v_j$ 
21: end if
22: return  $\pi'_1, \pi'_2, \pi'$ 

```

assignments, ensuring that each task is assigned to only one UAV. Next, the function constructs the set of unassigned tasks by comparing the current allocation to the global task set, and reassigns these tasks to UAVs with the lightest task load after random shuffling, thus improving both task coverage and load balance. Finally, the corrected solution is returned, ensuring that each task appears exactly once and that the overall task distribution is balanced. This lightweight yet effective repair mechanism guarantees assignment feasibility, avoids UAV overload or task omission, and supports robust downstream planning and execution. The hybrid mutation strategy refers to randomly selecting one mutation approach from the perturbation of the local neighborhood and task swapping to ensure population diversity. Local neighborhood perturbation involves randomly selecting a continuous subpath $\pi_{\text{sub}} = (v_i, v_{i+1}, \dots, v_j)$ within an individual π and then reoptimizing it using the 2-opt algorithm: $\pi'_{\text{sub}} = \arg \min_{\pi_{\text{sub}}} \sum_{k=i}^{j-1} d_{k, k+1}$. Intelligent task swapping, on the other hand, involves swapping tasks between different UAVs based on node distances.

- 3) *Elite Retention and Tournament Selection:* The collaborative mechanism of elite retention and tournament selection is employed to balance the convergence and diversity of the Pareto front. In the elite retention

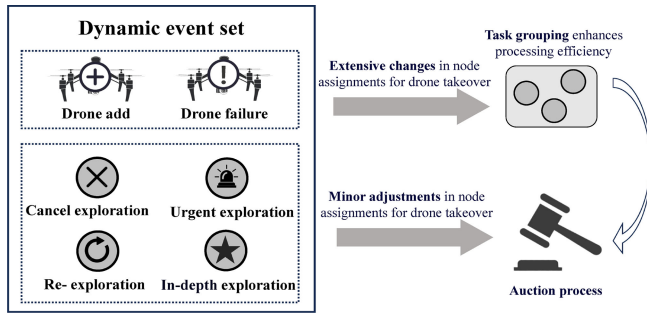


Fig. 5. Adaptive auction framework.

strategy, let the current population be P_t , the offspring population be Q_t , and the combined population be $R_t = P_t \cup Q_t$. Based on the nondominated sorting and crowding distance of multiobjective optimization, the top N_e best individuals are retained. The elite individuals $E_t = \{\pi \in R_t | \text{rank}(\pi) \leq r_{\text{elite}} \wedge \text{crowd}(\pi) \geq d_{\min}\}$, where $\text{rank}(\pi)$ is the nondominated sorting level of individual π , and $\text{crowd}(\pi)$ is the crowding distance. r_{elite} is the maximum nondominated level for elite retention, and $d_{\min}$ is the crowding distance threshold. From the remaining population, $N - N_e$ individuals are selected through a tournament selection mechanism. k individuals are randomly selected to form the tournament pool $T \subseteq R_t \setminus E_t$, and the individual with the lowest nondominated rank and highest crowding distance is chosen as $\pi^* = \arg \min_{\pi \in T} \text{rank}(\pi)$ if unique; otherwise, $\pi^* = \arg \max_{\pi \in T} \text{rank}(\pi)$. The next-generation population is formed by the elite individuals and the tournament-selected individuals

$$P_{t+1} = E_t \cup \{\pi_1^*, \pi_2^*, \dots, \pi_{N-N_e}^*\}. \quad (14)$$

B. Phase 2 (Distributed Dynamic Task Allocation)

In response to dynamic events, a distributed negotiation mechanism based on an adaptive auction algorithm is employed. This mechanism enables local rapid decision-making through bidding and auction rules, ensuring real-time performance in dynamic scenarios and allowing for flexible resource adjustments to adapt to environmental changes. The two-phase approach, combining initial optimization of task sequences with dynamic auctions, forms a complete solution of “static global optimization - dynamic local adjustment.”

The auction model proposed in this study involves two main participants: 1) price follower (PF) and 2) price leader (PL). PF initiates dynamic events and bids based on the scale of the task nodes that need to be handled. After communication within a ring network, PF, the initiator of the auction, determines the highest bidder as the winner and announces the auction results, as shown in Fig. 5.

As summarized in Table I, the dynamic events considered in this study are categorized into two major types: UAV events and node events. UAV events encompass Drone Failure and Drone Addition, both of which necessitate topology

updates and trigger auction-based task reallocation. In the case of a drone failure, the subsequent UAV in the communication topology detects the fault and initiates an auction to redistribute the tasks previously assigned to the failed UAV among the remaining agents. For drone addition, the newly joined UAV does not trigger a full redistribution of tasks. Instead, an auction is initiated only if more than two tasks remain in the system, with the auction focusing on reallocating the final one-third segment of the current task sequences. This ensures that the new UAV contributes to the remaining workload while minimizing disruption to already assigned tasks. The auction is conducted by all UAVs except the newly added one, guaranteeing that at least one task is redistributed. No auction occurs if only two tasks remain.

Node events include Cancel Exploration, Urgent Exploration, Re-exploration, and In-depth Exploration, which are typically managed locally without altering the communication topology. Among these, only Re-exploration triggers an auction, initiated by the UAV closest to the affected node. The remaining node events are handled directly by the UAV responsible for the corresponding task, without requiring any global coordination or topology modifications.

The specific impact of different events on system simulation is as follows: a UAV failure event randomly selects an active UAV in the system and sets it to a failed state; a UAV addition event randomly selects a new UAV at the starting point in the system; a key node exploration event increases the exploration time for the corresponding task node; a node cancellation event either cancels the exploration of that node or marks the node as an obstacle; a node priority exploration event immediately triggers exploration of that node; a node re-exploration event requires re-exploring the corresponding task node.

1) *UAV Swarm Network Topology*: We model the UAV swarm communication topology as a directed ring to simplify the exchange of dynamic-event notifications and auction information, as detailed communication protocols are beyond the scope of this study. For simplicity, we assume a fully connected network with ideal transmission conditions for all UAVs in this model. As illustrated in Fig. 6, let n UAVs form a ring topology, represented by the directed graph $G' = (V', E')$, where $V' = V_1, V_2, \dots, V_n$ is the set of vertices denoting the UAVs, and $E' \subseteq V \times V$ is the edge set representing the unidirectional communication links within the ring. As UAV failures or additions occur, the topology is updated through local ring reconstruction to maintain connectivity. The dual-ring topology is composed of two distinct rings: the topology ring and the auction data ring, as shown in Fig. 6. The topology ring is responsible for maintaining the communication network structure and handling dynamic changes, such as UAV failures or additions, with local ring reconstruction ensuring continuous connectivity. On the other hand, the auction data ring is dedicated to managing task reallocation and auction information exchange among UAVs. This separation of roles between the two rings enhances network

TABLE I
EVENT CATEGORIES AND DESCRIPTIONS

Event Category	Event Name	Event Description	Message Receiver	Topology Update	Auction	Auction Initiator
UAV Events	Drone Failure	Task cessation due to UAV failure	Next UAV in the communication topology	Yes	Yes	Next UAV in the communication topology
	Drone Addition	Addition of a new UAV to collaborate in remaining tasks	UAV closest to the new UAV's take-off point	Yes	Yes	All UAVs except the newly added UAV
	Cancel Exploration	Node exploration cancellation	UAV responsible for the node	No	No	-
Node Events	Urgent Exploration	Node requiring prioritized exploration	UAV responsible for the node	No	No	-
	Re-exploration	Node requiring re-exploration	UAV closest to the node	No	Yes	UAV closest to the node
	In-depth Exploration	Node requiring extended exploration time	UAV responsible for the node	No	No	-

Algorithm 3 Adaptive Auction Algorithm

Input: Dynamic event e , UAV set U , ring communication topology E'

Output: Auction winner u_{winner} , task allocation result A

```

1: if  $e \in \{\text{drone\_failure}, \text{drone\_add}, \text{re\_explore}\}$  then
2:   AuctionMsg  $\leftarrow (e, \text{nodes}, t_{\text{deadline}})$ 
3:    $u_{\text{init}} \leftarrow \text{SelectInitiator}(U)$ 
4: else
5:   Task queue update
6: end if
7: //Auction initialization
8: if  $|\text{nodes}| > \text{threshold}$  then
9:   Groups  $\leftarrow \text{PartitionNodes}(\text{nodes}, \text{max\_size} = 3)$ 
10:  for each  $G \in \text{Groups}$  do
11:    RunAuction( $G$ )
12:  end for
13: else
14:  RunAuction( $\text{nodes}$ )
15: end if
16:  $u_i \cdot \text{Send}(\mathcal{T} \cdot \text{Next}(u_i), \text{AuctionMsg})$ 
17: //Bid calculation and transmission
18: for each  $u_i \in U \setminus \{u_{\text{init}}\}$  do
19:    $B_i \leftarrow \mathcal{B}_i^j$ 
20:    $u_i \cdot \text{Send}(\mathcal{T} \cdot \text{Next}(u_i), B_i)$ 
21: end for
22: //Winner determination and task allocation
23: Bids  $\leftarrow u_{\text{init}} \cdot \text{CollectBids}()$ 
24:  $u_{\text{winner}} \leftarrow \arg \max_{u_i \in U} B_i$ 
25:  $\mathcal{A} \leftarrow \text{AssignTasks}(u_{\text{winner}}, \text{nodes})$ 
26:  $u_i \cdot \text{Send}(\mathcal{T} \cdot \text{Next}(u_i), \text{AssignTasksMsg})$ 
27: return UpdateStatus( $u_{\text{winner}}, \mathcal{A}$ )

```

scalability and resilience, enabling continuous communication even under dynamic conditions. By isolating the network maintenance and task redistribution functions, the dual-ring design reduces message congestion and improves overall system performance.

- 2) *Dynamic Event Handling Mechanism:* In dynamic scenarios, such as UAV addition, failure, or task updates, the initiator UAV responsible for triggering task reallocation is selected based on a proximity-aware, event-driven strategy. Specifically, when a new UAV joins the system, the UAV closest to its starting position is chosen as the initiator. This UAV releases approximately one-third of its assigned tasks (ensuring that at least one task is released if more than two tasks are assigned) for auction, allowing the new UAV to quickly assume a meaningful

workload. This approach promotes fairness by integrating new agents through local task redistribution, rather than requiring global replanning. In the case of UAV failure, the next UAV in the predefined ring communication topology is selected as the initiator, enabling rapid and localized recovery of the failed UAV's tasks. For general dynamic task updates, such as the addition or relocation of tasks, the UAV closest to the affected task is chosen as the initiator. This context-sensitive initiator selection enhances system adaptiveness by ensuring that task redistribution occurs with minimal coordination latency and localized communication. Furthermore, this mechanism improves the practical deployability of the system in real-world multi-UAV applications by reducing dependence on centralized decision-making, decreasing communication overhead, and better aligning with distributed swarm architectures. The PL is represented by a set of dynamic events, $\text{new}(\text{posnew}(x, y), c, t)$, where $\text{posnew}(x, y)$ denotes the event location, $c = (\text{event}, \text{node})$ specifies the event type and the number of affected nodes, and t indicates the time of event occurrence. Or nodes requiring reassignment of executing UAVs, each potential PF $PF_i \in \text{UAV}$ places a bid according to (15)–(17). The task insertion cost function (17) evaluates the additional time required to insert a new task into the UAV's task sequence. It consists of two components: insertion time cost, calculated by determining the best insertion position for the new task, evaluating the path cost between the current and new task, and dividing by the UAV's speed; and exploration time cost, which represents the predefined time required for the UAV to complete the new task. The total insertion cost is the sum of these two components. The UAV selects the position that minimizes the total insertion cost. All UAVs send their bids to the initiating UAV, which acts as the auctioneer. The initiating UAV selects the highest bidder as the winner and broadcasts the auction result to the swarm via the ring topology

$$B_i = (\mathcal{B}_i^1, \mathcal{B}_i^2, \dots, \mathcal{B}_i^{\text{node}}) \quad (15)$$

$$\mathcal{B}_i^j = B - \text{cost}, \quad j = 1, 2, \dots, \text{node} \quad (16)$$

$$\text{cost} = \text{cost}^{\text{insert}} + \text{cost}^{\text{current}} = \alpha^{\text{insert}} \cdot t^{\text{insert}} + \alpha^{\text{task}} \cdot t. \quad (17)$$

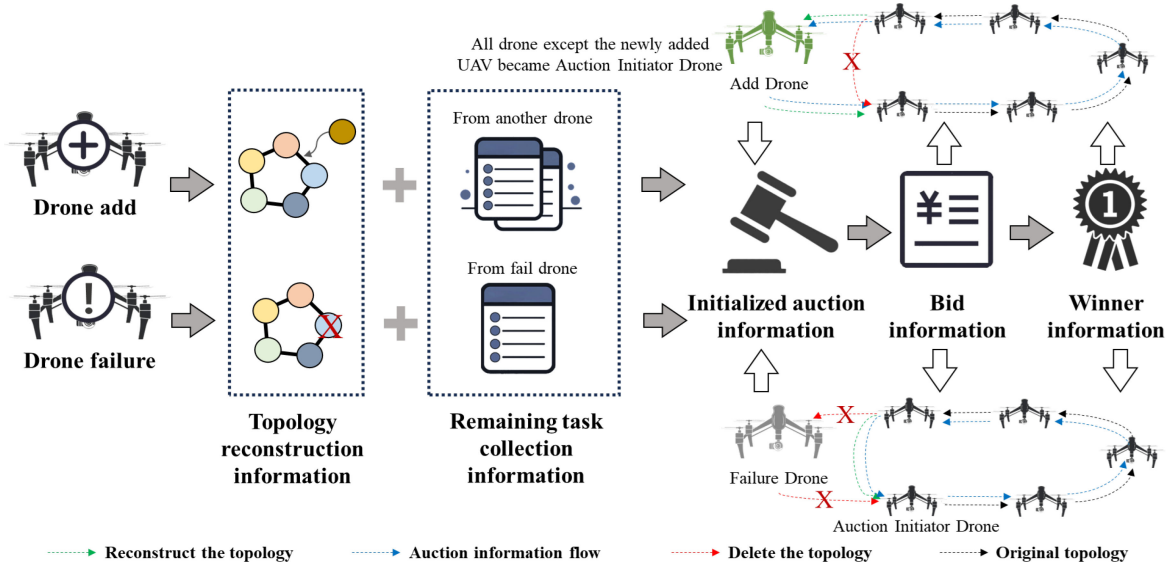


Fig. 6. Information flow in the UAV ring topology.

Here, $\mathcal{B}_i^j$ denotes the maximum bid placed by UAV PF_i for node j . The base score B is set to 100. The bidding cost represents the UAV's cost of inserting the task and consists of two components: the insertion cost $\text{cost}^{\text{insert}}$ and the current task cost $\text{cost}^{\text{current}}$. These are defined in terms of the additional time t^{insert} incurred by inserting the new node and the execution time t of the UAV's existing task sequence. The weights α^{insert} and α^{task} are assigned equal importance, indicating that both the insertion time cost and the current task execution time are considered of comparable significance. These weights, however, can be adjusted depending on the specific optimization objectives. For instance, they can be fine-tuned to either minimize overall task completion time or achieve a more balanced workload distribution across UAVs.

V. EXPERIMENTAL SETUP AND RESULTS ANALYSIS

A. Simulation Environment Setup

The experimental environment used in this study includes PyCharm Professional Edition 2024.2.4 and Python 3.12, running on a PC with Ubuntu 20.04, equipped with an Intel Core i9-14900HX $\times 32$ processor and 32 GB of RAM. The proposed improved NSGA-II algorithm was validated and benchmarked against both the PSO-GA hybrid algorithm and the standard NSGA algorithm. A schematic of the coverage task is illustrated in Fig. 7, where the blue node represents the starting point, red nodes indicate obstacles, and the remaining nodes are designated task points. UAVs begin their mission from the starting point and are required to fully cover the entire area by exploring all task nodes.

To verify the robustness of the algorithm, random starting points and obstacles are generated. In the simulation, the drones are randomly assigned starting points. The simulation parameters for different scales are shown in Table II, which is divided into small-scale and large-scale experiments. The

TABLE II
SIMULATION PARAMETER SETTINGS

Parameter (Unit)	Small-Scale Experiment	Large-Scale Experiment	Very Large-Scale Experiments
Number of Starting Points (Count)	3	5	10
Number of Drones (Count)	5	15	50
Number of Obstacles (Count)	15	30	100
Scale of Coverage Task Nodes (Count)	82	370	800
Maximum Simulation Time (s)	150	500	1000
Simulation Step Size (s)	0.1	0.5	5
α in Eq. (10)	0.5	0.5	0.5
α^{insert} in Eq. (17)	1	1	1
α^{task} in Eq. (17)	1	1	1

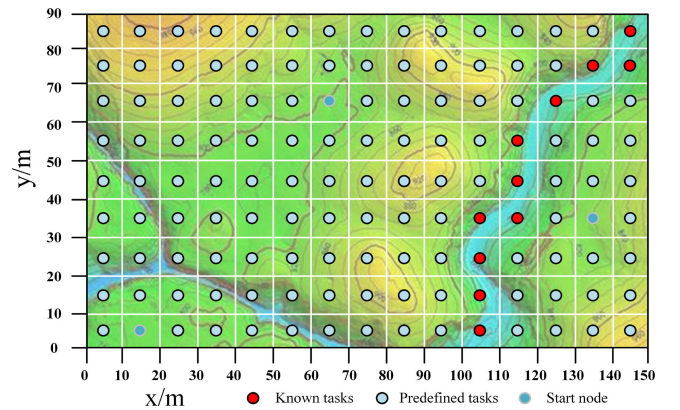


Fig. 7. Information flow in the UAV ring topology.

drones can move in eight directions. Table II also presents the values of several key weighting parameters used in the algorithm, including α in (10) and α^{insert} , α^{task} in (17). In this study, these weights are uniformly set at default values: $\alpha = 0.5$ and $\alpha^{\text{insert}} = \alpha^{\text{task}} = 1.0$, reflecting a neutral tradeoff between competing objectives, such as balancing time cost and workload. It is important to note that the design and tuning

TABLE III
EXPERIMENTAL RESULTS OF ALGORITHM COMPARISON

Algorithm	200 Iterations (Mean $\pm$ SD over 50 runs)					
	Small-scale		Large-scale		Very Large-scale	
	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)
PSO-GA	282.73 $\pm$ 21.91	82.20 $\pm$ 18.15	1433.02 $\pm$ 121.19	165.58 $\pm$ 27.53	3304.12 $\pm$ 210.14	141.34 $\pm$ 30.90
NSGA-II	214.97 $\pm$ 11.90	48.88 $\pm$ 4.15	1109.85 $\pm$ 73.18	89.97 $\pm$6.94	3667.10 $\pm$ 295.12	107.20 $\pm$10.27
Our method	214.60 $\pm$16.30	47.17 $\pm$5.82	1099.44 $\pm$186.54	96.61 $\pm$ 27.41	3069.93 $\pm$410.12	109.61 $\pm$ 25.90
Algorithm	500 Iterations (Mean $\pm$ SD over 50 runs)					
	Small-scale		Large-scale		Very Large-scale	
	Total Time a(s)	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)
PSO-GA	272.50 $\pm$ 21.75	77.61 $\pm$ 12.01	1414.42 $\pm$ 103.36	167.39 $\pm$ 26.83	3335.99 $\pm$ 179.94	139.11 $\pm$ 22.74
NSGA-II	217.55 $\pm$ 14.86	49.64 $\pm$ 4.18	1103.61 $\pm$ 69.73	91.13 $\pm$ 7.22	3520.59 $\pm$ 222.35	104.63 $\pm$ 10.15
Our method	214.68 $\pm$16.21	46.65 $\pm$3.19	1056.37 $\pm$92.89	86.61 $\pm$6.98	3000.48 $\pm$441.72	104.55 $\pm$26.34
Algorithm	1000 Iterations (Mean $\pm$ SD over 50 runs)					
	Small-scale		Large-scale		Very Large-scale	
	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)
PSO-GA	280.50 $\pm$ 24.57	82.88 $\pm$ 14.59	1476.50 $\pm$ 100.00	175.21 $\pm$ 28.10	3297.26 $\pm$ 187.09	137.60 $\pm$ 18.13
NSGA-II	220.41 $\pm$ 14.91	50.38 $\pm$ 4.37	1110.37 $\pm$ 65.47	91.45 $\pm$ 7.84	3567.67 $\pm$ 181.37	105.03 $\pm$ 7.79
Our method	217.25 $\pm$16.87	47.28 $\pm$3.14	1043.79 $\pm$102.10	86.62 $\pm$5.67	2979.60 $\pm$296.80	104.92 $\pm$26.37

of these parameters are not the focus of this article. Rather, they are intended to serve as generalizable and adjustable components within the framework that can be adapted to different mission requirements in future applications. The current setting provides a consistent baseline across problem scales for evaluating the algorithm's overall performance.

B. Comparison of Centralized Static Task Allocation Experiments

Based on the simulation environment described above, the experiment is conducted in three iterative stages (200, 500, and 1000 generations), with each configuration executed 50 times to eliminate the influence of randomness. To systematically evaluate the algorithm's performance, the algorithm is compared with existing classical hybrid algorithms, such as the PSO-GA algorithm and the traditional NSGA-II algorithm. The optimization quality is assessed through total execution time, maximum execution time, and the hypervolume (HV) metric to quantify the algorithm's performance. The results are shown in Table III. Here, total execution time refers to the sum of the execution times for all drones' task sequences, while maximum execution time refers to the maximum execution time across all drones' task sequences.

Table IV presents a comparative analysis of the HV indicator achieved by NSGA-II and the proposed method under different interaction settings and problem scales. Across all experimental settings, small-scale, large-scale, and very large-scale, the proposed method significantly outperforms NSGA-II in terms of HV values, indicating a better approximation to the Pareto front. For instance, in the small-scale experiment, the HV of the proposed method reaches 422.842, 494.453, and 522.808 for interaction settings of 200, 500, and 1000, respectively. In contrast, NSGA-II achieves considerably lower HV values under the same conditions. This trend becomes even more pronounced as the problem scale increases. In very large-scale experiments, the proposed method maintains

TABLE IV
COMPARISON OF ALGORITHMS BASED ON THE HV INDICATOR

Algorithm	Interaction (Generation)	Small-scale Experiment	Large-scale Experiment	Very Large-scale Experiment
NSGA-II	200	218.237	1790.680	4595.707
	500	228.179	2150.145	3930.922
	1000	239.517	1797.322	3862.109
Our method	200	422.842	5167.690	57843.298
	500	494.453	5719.295	46994.255
	1000	522.808	5400.098	45292.156

a strong performance, with the HV peaking at 57843.298 when the interaction is set to 200, which is more than 12 times greater than the corresponding value obtained by NSGA-II. These results demonstrate that the proposed method not only maintains robustness across different levels of interaction but also scales effectively with problem complexity. The substantial improvements in HV highlight the method's ability to generate a diverse and well-distributed set of Pareto-optimal solutions, especially in high-dimensional and large-scale scenarios.

The experiments demonstrate that the proposed method exhibits significant advantages in terms of solution quality. As shown in Fig. 8, in small-scale experiments, compared to traditional NSGA-II and PSO-GA algorithms, the proposed method performs better in both the maximum execution time and total execution time metrics. Specifically, in the 200-generation iteration experiment, the proposed method achieved a maximum execution time of 47.17 $\pm$ 5.82, which is 3.5% lower than the traditional NSGA-II and 42.62% lower than PSO-GA. The total execution time was 214.60 $\pm$ 16.30, which is 0.17% lower than NSGA-II and 24.10% lower than PSO-GA. As the iteration count increased to 1000 generations, the efficiency advantage of the proposed method further expanded, with total execution time reduced by an average of 1.54% compared to NSGA-II, and maximum execution time reduced by an average of 6.09%, indicating that the growth rate of its time complexity is significantly lower than that of the comparison

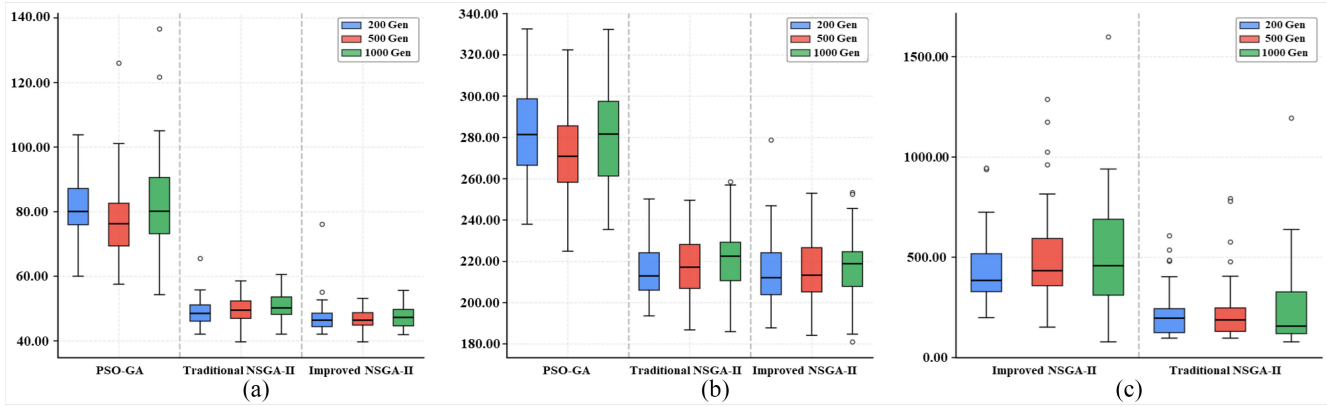


Fig. 8. Small-scale experiment results. (a) Maximum execution time. (b) Total execution time. (c) HV.

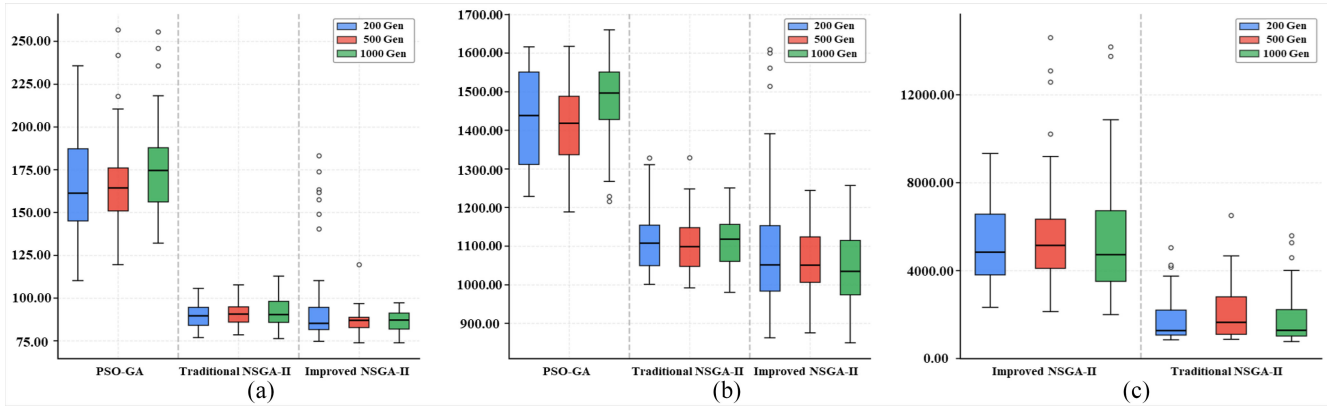


Fig. 9. Large-scale experiment results. (a) Maximum execution time. (b) Total execution time. (c) HV.

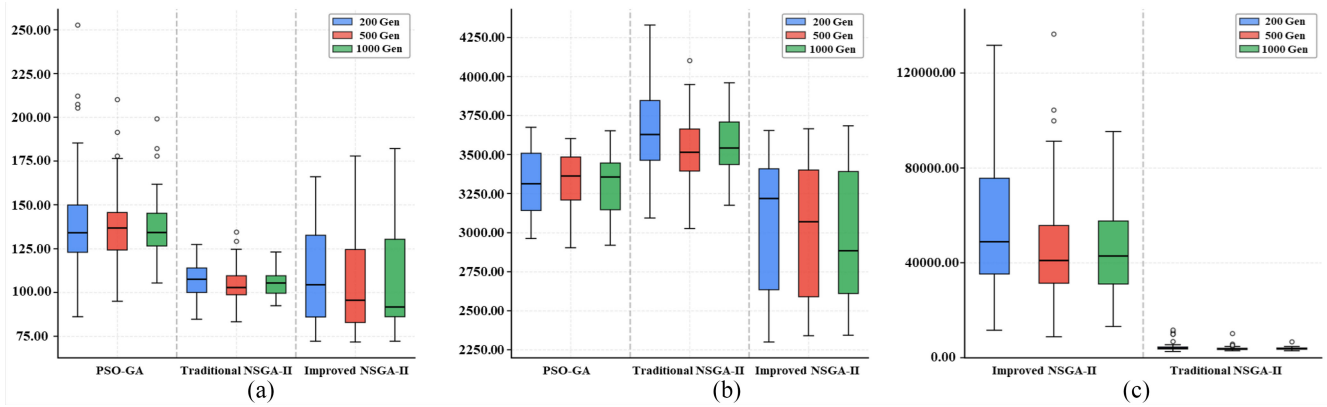


Fig. 10. Very large-scale experiment results. (a) Maximum execution time. (b) Total execution time. (c) HV.

algorithms. In terms of solution quality, the HV indicator validates that the proposed method dominates the traditional NSGA-II across all iteration counts. In the 200-generation experiment, its HV value surpassed that of the comparison algorithms, and as the iteration count increased, this advantage continued to grow, demonstrating that the nondominated solution set generated by the proposed method covers a larger volume in the objective space, with better distribution and convergence.

In the large-scale experiments, as shown in Fig. 9, the algorithm's total execution time was on average reduced by approximately 25.97% compared to the PSO-GA algorithm,

and by approximately 3.74% compared to the traditional NSGA-II algorithm. The time distribution was consistently concentrated in the 900–1100 s range. Notably, as the number of iterations gradually increased from 200 to 1000, an analysis of the standard deviations of the total execution time and maximum execution time indicated that the algorithm converged after 500 iterations, with output results stabilizing. In terms of solution quality, the algorithm's HV mean value was significantly higher than that of the traditional NSGA-II algorithm, with the adjustment strategies notably enhancing both computational efficiency and solution quality. Further analysis revealed that this advantage stemmed from improvements in

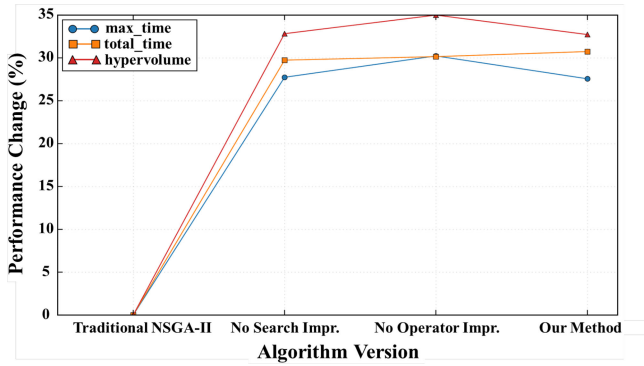


Fig. 11. Ablation study of proposed enhancements in performance.

the algorithm's population diversity preservation and global search capability, enabling it to effectively avoid premature convergence and explore the uncovered areas of the Pareto front.

In very large-scale experiments, the proposed method's optimization ability scales up with task complexity. As shown in Fig. 10, it achieves a 7.1% reduction in total execution time and 22.4% in maximum time at 200 iterations compared to PSO-GA. By 1000 iterations, these figures grow to 9.6% and 23.8%, respectively. This suggests strong scalability and increasing advantage as task environments become more complex. Across all scales, the proposed method demonstrates not only consistent performance but also increasing efficiency gains as the task environment grows in complexity. Its strong scalability makes it particularly suitable for large-scale real-world multiagent deployment scenarios.

To evaluate the individual contribution of each enhancement module within our proposed algorithm, we conduct ablation experiments under ultralarge-scale task scenarios with a fixed evolution budget of 1000 generations. Four algorithmic variants are examined.

- 1) The standard NSGA-II algorithm, serving as the baseline (Traditional NSGA-II).
- 2) NSGA-II incorporating the dual-population initialization and enhanced crossover/mutation operators, but excluding the path-exchange crossover module (No Search Improvements).
- 3) NSGA-II with the dual-population and path-exchange crossover, but employing conventional single-point crossover and Gaussian mutation (No Operator Improvements).
- 4) The fully enhanced version integrating all proposed components (Our method).

Simulation results demonstrate that each enhancement in the proposed method contributes distinctively to performance improvement as shown in Fig. 11. The removal of the search improvement module results in a 27.7% increase in maximum execution time and a 29.7% increase in total execution time, indicating its critical role in refining solution quality through effective local path optimization. In contrast, excluding the operator improvement module leads to a 30.2% increase in both metrics, underscoring its contribution to maintaining population diversity and preventing premature convergence

to local optima. While the operator module alone achieves the lowest maximum execution time (75.24 s), our integrated method delivers the best total execution time (2490.27 s), representing a 30.7% improvement over the baseline NSGA-II. This suggests that although a slight compromise in maximum execution time is observed (78.11 s), the combined strategy achieves a more balanced and robust optimization outcome. Overall, the synergy between enhanced search mechanisms and adaptive genetic operators allows our method to effectively explore and exploit the solution space, leading to superior performance across both objectives.

C. Robustness Analysis of Distributed Dynamic Task Allocation Algorithm

In the scalability and fault-tolerance verification experiments, this study introduces a multistage dynamic evaluation framework comprising progressive gradient expansion experiments (randomly adding 2, 4, or 8 UAVs) and corresponding gradient degradation experiments (randomly removing 2, 4, or 8 UAVs). This framework is designed to assess the adaptability of the auction-based task reallocation mechanism in dynamic environments.

As shown in Fig. 12, when the swarm size increases from the baseline group to 23 UAVs, the system maintains the maximum load difference among UAVs below 28%, demonstrating excellent scalability. In degradation scenarios, even when more than 50% of the UAVs (8 units) unexpectedly fail, the system successfully redistributes tasks among the remaining agents, restoring the maximum load difference to within 1.3 times its original value.

Crucially, once load balancing is restored, the system maintains a stable task completion rate, ensuring that all subtasks continue to be effectively reassigned and executed across the remaining UAVs. This indicates that task coverage remains intact and mission execution continues efficiently despite UAV failures. These results confirm the robustness and reliability of the proposed algorithm in handling dynamic swarm size scenarios, providing strong support for collaborative UAV task planning in complex operational environments.

To assess the effectiveness and robustness of the proposed method, we conducted extensive simulation experiments under dynamic task allocation scenarios, including UAV expansion and failure conditions. In the dynamic phase, we compared the full dual-ring scheduler with two baseline algorithms: Random Dynamic Allocation and the Pure Distributed Auction Algorithm. The Random Dynamic Allocation approach simulates a centralized system, where tasks are randomly assigned to UAVs without regard to proximity or workload. This method highlights the inefficiencies inherent in a centralized approach under dynamic conditions, where the absence of global information results in poor performance and potential system failure. The lack of global knowledge in this approach leads to inefficient task handling, which can result in errors or task allocation failures. In contrast, the Pure Distributed Auction Algorithm relies entirely on auction mechanisms for task assignment, excluding centralized planning. Unlike the full dual-ring scheduler, it uses only the auction process

TABLE V
ABLATION STUDY RESULTS

Algorithm	Max Time (s)		Total Time (s)		HV
	Mean $\pm$ SD over 50 runs	Improvement	Mean $\pm$ SD over 50 runs	Improvement	
Traditional NSGA-II	107.84 $\pm$ 11.09	—	3595.65 $\pm$ 225.43	—	3927.09
No Search Improvements	77.92 $\pm$ 5.74	27.7%	2526.25 $\pm$ 181.15	29.7%	62873.94
No Operator Improvements	75.24 $\pm$ 4.51	30.2%	2511.45 $\pm$ 80.92	30.2%	66787.77
Our method	78.11 $\pm$ 5.91	27.6%	2490.27 $\pm$ 85.02	30.7%	62693.81

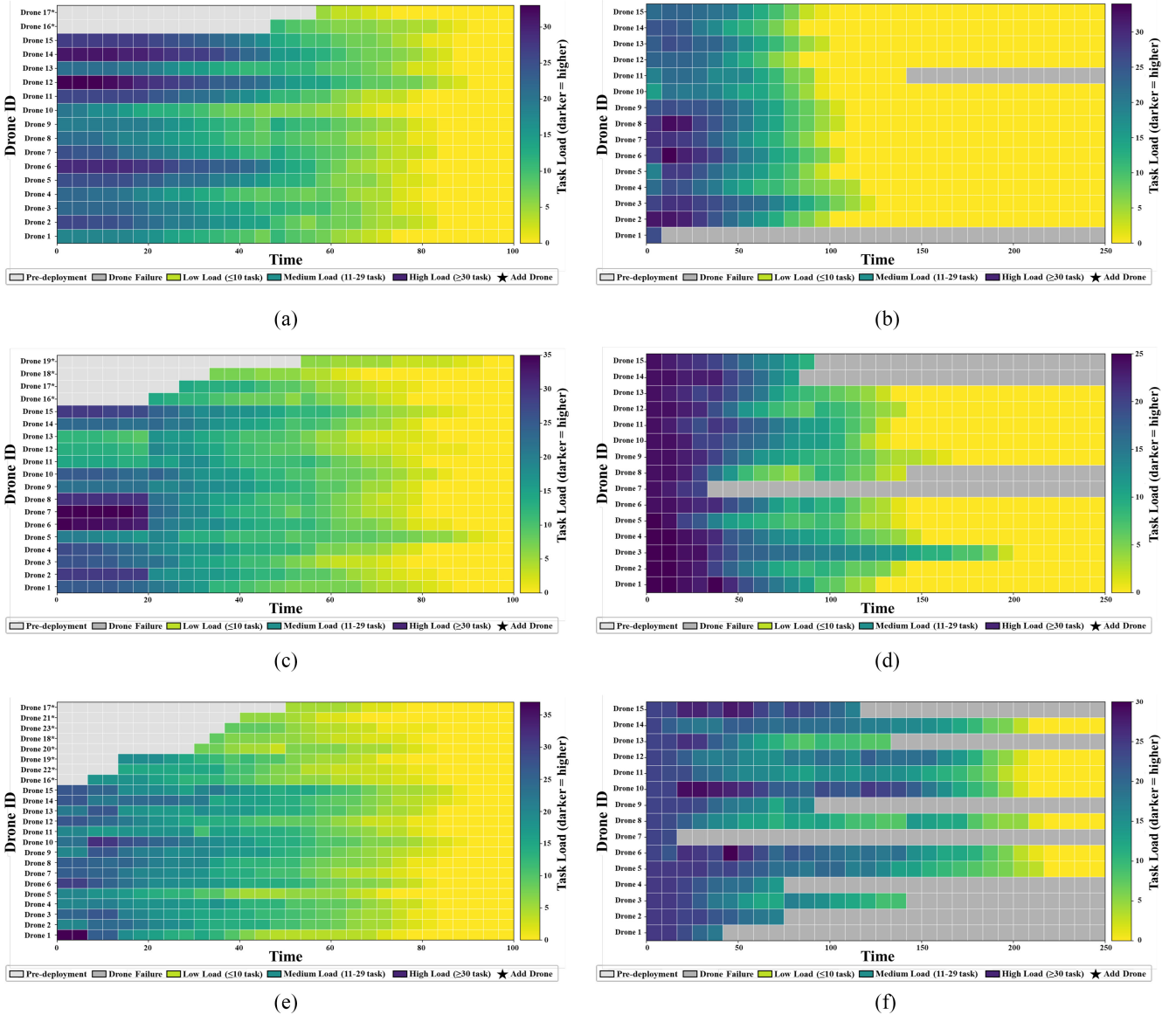


Fig. 12. UAV Dynamic Event Timeline - Load Heatmap. (a) Randomly adding two UAVs. (b) Randomly removing two UAVs. (c) Randomly adding four UAVs. (d) Randomly removing four UAVs. (e) Randomly adding eight UAVs. (f) Randomly removing two UAVs.

for both initial task allocation and task redistribution. All algorithms were tested across three different UAV fleet sizes (2, 4, and 8 UAVs) to simulate varying levels of system complexity. The experimental results, presented in Fig. 13 and Table V, clearly demonstrate the effectiveness and advantages of the hybrid approach, particularly in terms of system robustness and task allocation efficiency across varying UAV fleet sizes.

To reflect real-world dynamics, two experimental scenarios were designed: 1) UAV expansion, in which additional UAVs are incrementally introduced during execution, and 2) UAV failure, where a subset of UAVs becomes nonfunctional partway through the task. Each configuration was repeated 50 times independently to ensure statistical reliability. The evaluation focuses on two key performance metrics: maximum execution time (i.e., the longest time taken by any single UAV)

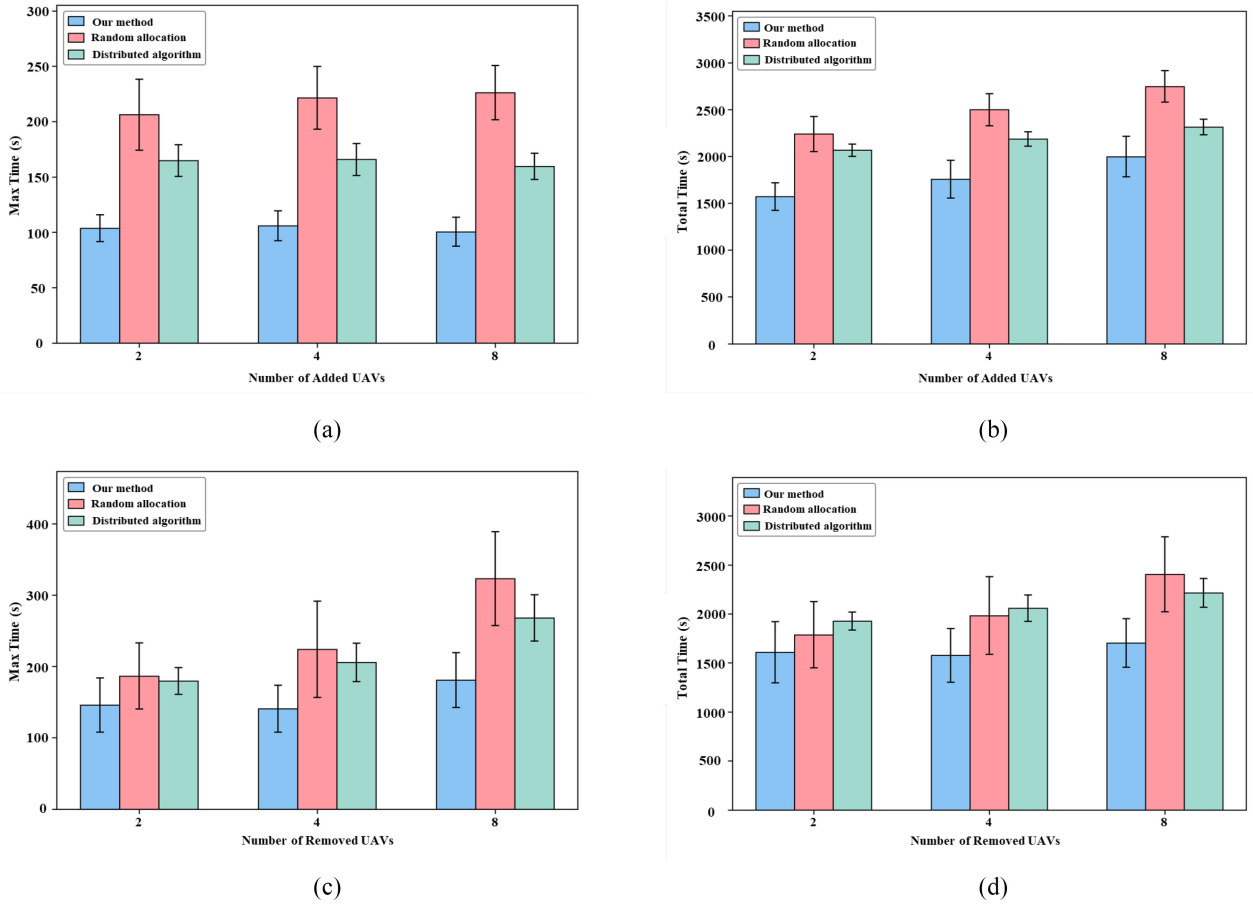


Fig. 13. Results of Gradient-Based Dynamic Testing. (a) Max execution time under expansion scenarios. (b) Total execution time under expansion scenarios. (c) Max execution time under failure scenario. (d) Total execution time under failure scenarios.

TABLE VI
EXECUTION TIME COMPARISON UNDER UAV EXPANSION AND FAILURE SCENARIOS

Algorithm	Expansion Scenarios (Mean $\pm$ SD over 50 runs)					
	2 UAVs		4 UAVs		8 UAVs	
	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)
Random Allocation	206.26 $\pm$ 32.03	2238.55 $\pm$ 187.99	165.79 $\pm$ 14.43	2186.11 $\pm$ 77.14	226.12 $\pm$ 24.49	2746.45 $\pm$ 168.56
Distributed Algorithm	164.79 $\pm$ 14.35	2065.56 $\pm$ 64.82	221.52 $\pm$ 28.36	2497.78 $\pm$ 172.64	159.54 $\pm$ 11.93	2314.40 $\pm$ 83.80
Our Method	103.60 $\pm$ 12.06	1569.81 $\pm$ 146.93	105.87 $\pm$ 13.42	1755.97 $\pm$ 202.14	100.37 $\pm$ 13.07	1997.39 $\pm$ 216.23
Algorithm	Failure Scenarios (Mean $\pm$ SD over 50 runs)					
	2 UAVs		4 UAVs		8 UAVs	
	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)	Max Time (s)	Total Time (s)
Random Allocation	186.54 $\pm$ 46.43	1786.91 $\pm$ 338.86	223.95 $\pm$ 67.63	1983.32 $\pm$ 397.52	323.07 $\pm$ 65.67	2404.58 $\pm$ 382.06
Distributed Algorithm	179.60 $\pm$ 18.68	1926.81 $\pm$ 90.67	205.57 $\pm$ 26.85	2057.92 $\pm$ 134.92	268.02 $\pm$ 32.34	2214.43 $\pm$ 148.13
Our Method	145.80 $\pm$ 37.83	1608.69 $\pm$ 313.62	140.68 $\pm$ 32.84	1577.13 $\pm$ 274.81	180.71 $\pm$ 38.46	1703.55 $\pm$ 248.12

and total execution time (i.e., the cumulative time for all tasks), with all results reported as mean $\pm$ standard deviation. These comparisons allowed us to assess the relative performance of our method in dynamic environments involving UAV expansion and failure. The final experimental results are presented in Table VI.

As shown in Fig. 13, the comparison of execution times in both UAV expansion and failure scenarios demonstrates that our method consistently outperforms both the Random Allocation and Distributed Algorithm in all test conditions. In the expansion scenario, with 2, 4, and 8 UAVs, our method

shows a significant reduction in both maximum and total execution times compared to the other two algorithms. For example, in the case of 2 UAVs, the maximum time for our method is 103.60 ± 12.06 , which is nearly half the time taken by Random Allocation (206.26 ± 32.03). As the number of UAVs increases, our method continues to maintain superior performance, with its total execution time remaining the lowest, even under the more complex 8 UAV scenario.

Similarly, in the failure scenario, our method again demonstrates its robustness by providing the shortest execution times. The maximum time for 2 UAVs is reduced by approximately

21. 8% compared to random allocation and 18. 9% compared to the Distributed Algorithm. The total improvement in execution time is similarly substantial, with a reduction of 10 to 16% compared to the other two algorithms in different counts of UAV. Overall, the results confirm that not only is our method more efficient, but also exhibits better scalability and stability under varying conditions.

VI. CONCLUSION

This study constructs a collaborative optimization framework for centralized static allocation and distributed dynamic adjustment, effectively integrating the dual advantages of global optimization and real-time response. Experimental verification demonstrates that the proposed framework reduces system task time by an average of 3.85%, while maintaining load balance in dynamic environments through the task reassignment mechanism. For multiobjective optimization problems, the improved NSGA-II algorithm, incorporating dual-population initialization, path exchange crossover, and hybrid mutation strategies, enhances the HV indicator by 49.7%. Compared to the PSO-GA algorithm and traditional NSGA-II, the total execution time is reduced by an average of 13.33%, and the maximum execution time is reduced by an average of 23.71%. The core innovation lies in combining prior knowledge to guide the search for the solution space with an explicit path constraint mechanism, providing a scalable hybrid architectural paradigm for cooperative optimization of unmanned systems in dynamic environments. While the proposed framework shows significant promise in simulation, practical deployment in real-world environments requires addressing challenges, such as communication latency, packet loss, and terrain-related constraints. These factors may negatively impact system responsiveness and coordination performance, necessitating careful consideration to ensure the system operates effectively in dynamic, real-world applications.

Building on this foundation, future research could focus on enhancing the robustness and quality of the dynamic allocation process, particularly in large-scale, rapidly changing environments. Moreover, while the current task model assumes sufficient resources for all UAVs and fixed task durations, these assumptions may not hold in practical settings, where UAVs face energy constraints, task complexity variations, and communication degradation. Future work should explore the relaxation of these assumptions by integrating energy constraints, variable task complexities, and communication disruptions into the model. Developing adaptive methods that account for these real-world factors will improve the system's applicability to actual operational scenarios. Additionally, incorporating advanced machine learning techniques, such as reinforcement learning and graph neural networks, could further enhance the adaptability and intelligence of the proposed framework. Finally, exploring cooperative strategies among heterogeneous agents, including coordination between UAVs, UGVs, and USVs, will open new avenues for addressing complex task allocation scenarios. These future directions offer substantial potential for advancing the practical deployment

and operational efficiency of multiagent systems in real-world environments.

REFERENCES

- [1] I. A. Shah, A. Laraib, H. Ashraf, and F. Hussain, "Drone technology: Current challenges and opportunities," *Cybersecurity Issues Challenges Drone Industry*. Hershey, PA, USA: IGI Global, 2024, pp. 343–361,
- [2] J. Aliloo, E. Abbasi, E. Karamidehkordi, E. G. Parmehr, and M. Canavari, "Dos and don'ts of using drone technology in the crop fields," *Technol. Soc.*, vol. 76, Mar. 2024, Art. no. 102456.
- [3] O. Dukkanci, J. F. Campbell, and B. Y. Kara, "Facility location decisions for drone delivery with riding: A literature review," *Comput. Oper. Res.*, vol. 316, no. 2, 2024, Art. no. 106672.
- [4] A. Lee, S.-P. Yong, W. Pedrycz, and J. Watada, "Testing a vision-based autonomous drone navigation model in a forest environment," *Algorithms*, vol. 17, no. 4, p. 139, 2024.
- [5] Y. Luo et al., "Star-searcher: A complete and efficient aerial system for autonomous target search in complex unknown environments," *IEEE Robot. Autom. Lett.*, vol. 9, no. 5, pp. 4329–4336, May 2024.
- [6] N. Ejaz and S. Choudhury, "Computer vision in drone imagery for infrastructure management," *Autom. Constr.*, vol. 163, Jul. 2024, Art. no. 105418.
- [7] K. AL-Dosari, Z. Hunaiti, and W. Balachandran, "Drone safety and security surveillance system (D4S)," *Appl. Sci.*, vol. 14, no. 10, p. 4193, 2024.
- [8] S. Xing, X. Zhang, J. Tian, C. Xie, Z. Chen, and J. Sun, "Morphing quadrotors: Enhancing versatility and adaptability in drone applications—A review," *Drones*, vol. 8, no. 12, p. 762, 2024.
- [9] D. Askerbekov, J. A. Garza-Reyes, R. R. Ghatak, R. Joshi, J. Kandasamy, and D. L. de Mattos Nascimento, "Embracing drones and the internet of drones systems in manufacturing—An exploration of obstacles," *Technol. Soc.*, vol. 78, Sep. 2024, Art. no. 102648.
- [10] K. Zhichkin, V. Nosov, L. Zhichkina, O. Anichkina, I. Borodina, and A. Bekeotov, "Efficiency of using drones in agricultural production," in *Proc. E3S Web Conf.*, vol. 381, 2023, p. 1048.
- [11] Z. Liu et al., "Graph neural network meets multi-agent reinforcement learning: Fundamentals, applications, and future directions," *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 39–47, Dec. 2024.
- [12] Y. Feng, F. Wang, Z. Liu, and Z. Chen, "An online topology control method for multi-agent systems: Distributed topology reconfiguration," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4358–4370, Sep./Oct. 2024.
- [13] P. Yadav, A. Mishra, and S. Kim, "A comprehensive survey on multi-agent reinforcement learning for connected and automated vehicles," *Sensors*, vol. 23, no. 10, p. 4710, 2023.
- [14] Y. Quan and L. Xi, "Smart generation system: A decentralized multi-agent control architecture based on improved consensus algorithm for generation command dispatch of sustainable energy systems," *Appl. Energy*, vol. 365, Jul. 2024, Art. no. 123209.
- [15] G. Zhang et al., "G-Designer: Architecting multi-agent communication topologies via graph neural networks," 2024, *arXiv:2410.11782*.
- [16] J. Qian, J. Wang, X. Li, and S. Jin, "UAV-MIMO under wobbling: A comparative analysis of Centralized and distributed implementations," *IEEE Trans. Commun.*, vol. 73, no. 8, pp. 6825–6839, Aug. 2025.
- [17] P. Cao et al., "Computational intelligence algorithms for UAV swarm networking and collaboration: A comprehensive survey and future directions," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2684–2728, 4th Quart., 2024.
- [18] X. Hai, Q. Feng, W. Chen, C. Wen, and A. W. Khong, "Capability-oriented decision-making in multi-uav deployment and task allocation: A hierarchical game-based framework," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 7, pp. 4562–4574, Jul. 2025.
- [19] M. Höffmann, S. Patel, and C. Büskens, "Optimal guidance track generation for precision agriculture: A review of coverage path planning techniques," *J. Field Robot.*, vol. 41, no. 3, pp. 823–844, 2024.
- [20] H. Wen, Y. Shi, S. Wang, T. Chen, P. Di, and L. Yang, "Route planning for UAVs maritime search and rescue considering the targets moving situation," *Ocean Eng.*, vol. 310, Oct. 2024, Art. no. 118623.
- [21] Z. Chen et al., "Improved coverage path planning for indoor robots based on BIM and robotic configurations," *Autom. Constr.*, vol. 158, Feb. 2024, Art. no. 105160.
- [22] U. Duh, V. Shankar, and G. Kosec, "Discretization of non-uniform rational B-spline (NURBS) models for meshless isogeometric analysis," *J. Sci. Comput.*, vol. 100, no. 2, p. 51, 2024.

- [23] B. Jia et al., "Coverage path planning for IoUAVs with tiny machine learning in complex areas based on convex decomposition," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21103–21111, Jun. 2024.
- [24] S. Kazemdehbash, "Adaptive grid-based decomposition for UAV-based coverage path planning in maritime search and rescue," 2024, *arXiv:2412.00899*.
- [25] H. Mazaheri, S. Goli, and A. Nourollah, "A survey of 3D space path-planning methods and algorithms," *ACM Comput. Surveys*, vol. 57, no. 1, pp. 1–32, 2024.
- [26] L. Huo et al., "Complete coverage path planning algorithm based on improved biologically inspired neural networks in spray painting," *IEEE Robot. Autom. Lett.*, vol. 10, no. 6, pp. 5697–5704, Jun. 2025.
- [27] S. Agarwal and S. Akella, "Line coverage with multiple robots: Algorithms and experiments," *IEEE Trans. Robot.*, vol. 40, pp. 1664–1683, 2024.
- [28] J. Luo, Y. Tian, and Z. Wang, "Research on unmanned aerial vehicle path planning," *Drones*, vol. 8, no. 2, p. 51, 2024.
- [29] K. L. Palmerius, A. Uggla, G. Fylkner, and J. Lundberg, "End-to-end drone route planning in flexible airspace design," *Transp. Res. Interdiscipl. Perspect.*, vol. 27, Sep. 2024, Art. no. 101219.
- [30] T. Zhou, Z. Zhou, H. Qiu, B. Niu, G. X.-G. Yue, and W. Pedrycz, "Two-stage knowledge-assisted coevolutionary NSGA-II for bi-objective path planning of multiple unmanned aerial vehicles," *Swarm Evol. Comput.*, vol. 90, Oct. 2024, Oct. 101680.
- [31] H. Cui, F. Cao, and R. Liu, "A multi-objective partitioning algorithm for large-scale graph based on NSGA-II," *Expert Syst. Appl.*, vol. 263, Mar. 2025, Art. no. 125756.
- [32] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.
- [33] E. W. Dijkstra, "A note on two problems in Connexion with graphs," in *Edsger Wybe Dijkstra: His Life, Work, and Legacy*. New York, NY, USA: Assoc. Comput. Mach., 2022, pp. 287–290.
- [34] S. M. LaValle and J. J. Kuffner, "Rapidly-exploring random trees: Progress and prospects," in *Algorithmic and Computational Robotics*. Boca Raton, FL, USA: CRC Press, 2001, pp. 303–307.
- [35] R. Sarkar, D. Barman, and N. Chowdhury, "Domain knowledge based genetic algorithms for mobile robot path planning having single and multiple targets," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 7, pp. 4269–4283, 2022.
- [36] M. Adamik, J. Goga, J. Pavlovicova, A. Babinec, and I. Sekaj, "Fast robotic pencil drawing based on image evolution by means of genetic algorithm," *Robot. Auton. Syst.*, vol. 148, Feb. 2022, Art. no. 103912.
- [37] P. Duan, Z. Yu, K. Gao, L. Meng, Y. Han, and F. Ye, "Solving the multi-objective path planning problem for mobile robot using an improved NSGA-II algorithm," *Swarm Evol. Comput.*, vol. 87, Jul. 2024, Art. no. 101576.
- [38] F. Yan, J. Chu, J. Hu, and X. Zhu, "Cooperative task allocation with simultaneous arrival and resource constraint for multi-UAV using a genetic algorithm," *Expert Syst. Appl.*, vol. 245, Jul. 2024, Art. no. 123023.
- [39] B. Yildiz, M. F. Aslan, A. Durdu, and A. Kayabasi, "Consensus-based virtual leader tracking swarm algorithm with GDRRT*-PSO for path-planning of multiple-UAVs," *Swarm Evol. Comput.*, vol. 88, Jul. 2024, Art. no. 101612.
- [40] B. Zhou, H. Xu, and S. Shen, "RACER: Rapid collaborative exploration with a decentralized multi-UAV system," *IEEE Trans. Robot.*, vol. 39, no. 3, pp. 1816–1835, Jun. 2023.
- [41] E. Yanmaz, H. M. Balanji, and İ. Güven, "Dynamic multi-UAV path planning for multi-target search and connectivity," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 10516–10528, Jul. 2024.
- [42] M. Rinaldi, S. Wang, R. S. Geronel, and S. Primates, "Application of task allocation algorithms in multi-UAV intelligent transportation systems: A critical review," *Big Data Cogn. Comput.*, vol. 8, no. 12, p. 177, 2024.
- [43] X. Wen, G. Wu, S. Li, and L. Wang, "Ensemble multi-objective optimization approach for heterogeneous drone delivery problem," *Expert Syst. Appl.*, vol. 249, Sep. 2024, Art. no. 123472.
- [44] R. De la Fuente, M. M. Aguayo, and C. Contreras-Bolton, "An optimization-based approach for an integrated forest fire monitoring system with multiple technologies and surveillance drones," *Eur. J. Oper. Res.*, vol. 313, no. 2, pp. 435–451, 2024.
- [45] M. Eser and A. E. Yilmaz, "A gossip-based auction algorithm for decentralized task rescheduling in heterogeneous drone swarms," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 61, no. 3, pp. 6673–6696, Jun. 2025.
- [46] X. Wang et al., "A survey on security of UAV swarm networks: Attacks and countermeasures," *ACM Comput. Surv.*, vol. 57, no. 3, pp. 1–37, 2024.
- [47] Z. Wang, J. Li, J. Li, and C. Liu, "A decentralized decision-making algorithm of UAV swarm with information fusion strategy," *Expert Syst. Appl.*, vol. 237, Mar. 2024, Art. no. 121444.
- [48] W. Kang, E. Jeong, S. Shim, and S. Ha, "Optimization of task allocation for resource-constrained swarm robots," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 3068–3085, 2025.
- [49] M. Sharma, A. Tomar, and A. Hazra, "Edge computing for industry 5.0: Fundamental, applications and research challenges," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19070–19093, Jun. 2024.
- [50] X. Ge, Q.-L. Han, X.-M. Zhang, D. Ding, and B. Ning, "Distributed coordination control of multi-agent systems under intermittent sampling and communication: A comprehensive survey," *Sci. China Inf. Sci.*, vol. 68, no. 5, 2025, Art. no. 151201.
- [51] J. Jiang, Y. Dai, F. Yang, and Z. Ma, "A multi-visit flexible-docking vehicle routing problem with drones for simultaneous pickup and delivery services," *Eur. J. Oper. Res.*, vol. 312, no. 1, pp. 125–137, 2024.
- [52] X. Li, Z. Zhang, W. Sun, Y. Liu, and J. Tang, "Parallel dynamic NSGA-II with multi-population search for rescheduling of seru production considering schedule changes under different dynamic events," *Expert Syst. Appl.*, vol. 238, 2024, Art. no. 121993.
- [53] X. Mao, G. Wu, M. Fan, Z. Cao, and W. Pedrycz, "DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 1028–1044, 2024.
- [54] D. Luo, Z. Fan, Z. Yang, and Y. Xu, "Multi-UAV cooperative maneuver decision-making for pursuit-evasion using improved MADRL," *Defence Technol.*, vol. 35, pp. 187–197, May 2024.
- [55] D. Kong, X. Li, P. Ni, Y. Hu, J. Hu, and W. Hu, "TopSPR-net: Topology aware segment-level point cloud learning descriptors for three-dimensional place recognition in large-scale environments," *IEEE Trans. Ind. Electron.*, vol. 71, no. 10, pp. 13406–13416, Oct. 2024.
- [56] G. Wang, J. Peng, C. Guan, J. Chen, and B. Guo, "Multi-drone collaborative shepherding through multi-task reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 10311–10318, Nov. 2024.
- [57] J. Amendola, L. R. Cenkeramaddi, and A. Jha, "Drone landing and reinforcement learning: State-of-art, challenges and opportunities," *IEEE Open J. Intell. Transp. Syst.*, vol. 5, pp. 520–539, 2024.
- [58] N. Dong, S. Liu, and X. Mai, "Communication-efficient heterogeneous multi-UAV task allocation based on clustering," *Comput. Commun.*, vol. 229, Jan. 2025, Art. no. 107986.



Yongzhuo Yu received the master's degree from Central South University, Changsha, China, in 2024. She is currently pursuing the Ph.D. degree with Beihang University, Beijing, China.

Her research interests primarily revolve around autonomous driving, mission planning, and unmanned systems, with a focus on their theoretical advancements and practical engineering implementations in intelligent transportation, collaborative unmanned operations, and cross-scenario autonomous technologies.



Xuting Duan received the Ph.D. degree in traffic information engineering and control from Beihang University, Beijing, China, in 2017.

He is currently a Professor with the School of Transportation Science and Engineering, Beihang University. His research interests are focused on the theories of V2X communications and collaborative intelligence, as well as their systematic engineering applications in 3-D intelligent transportation systems, connected and autonomous vehicles, and cross-domain collaborative unmanned systems.



Feiyang Zhao received the bachelor's degree in engineering from Jilin University, Changchun, China, in 2024. He is currently pursuing the master's degree with the School of Transportation Science and Engineering, Beihang University, Beijing, China.

His current research interests include multiagent collaboration, simultaneous localization and mapping, and autonomous driving.



Jianshan Zhou received the Ph.D. degree in traffic information engineering and control from Beihang University, Beijing, China, in 2020.

He is currently an Associate Professor with the School of Transportation Science and Engineering, Beihang University. His current research interests include wireless communication, artificial intelligent system, and intelligent transportation system.



Chunmian Lin (Member, IEEE) received the Ph.D. degree in electronics and information from Beihang University, Beijing, China, in 2023.

He is currently a Postdoctoral Fellow with the School of Transportation Science and Engineering, Beihang University. His research interests include autonomous driving, image processing, computer vision, information fusion, and deep learning, particularly their applications in intelligent transportation systems.



Kaige Qu (Member, IEEE) received the B.Sc. degree in communication engineering from Shandong University, Jinan, China, in 2013, the first M.Sc. degree in integrated circuits engineering from Tsinghua University, Beijing, China, the second M.Sc. degree in electrical engineering from KULeuven, Leuven, Belgium, in 2016, and the Ph.D. degree in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada, in 2021.

From 2021 to 2023, she was a Postdoctoral Fellow and then a Research Associate with the Department of Electrical and Computer Engineering, University of Waterloo. Her research interests include connected and autonomous vehicles, network intelligence, network virtualization, and digital twin assisted network automation.



Daxin Tian (Fellow, IEEE) received the Ph.D. degree in computer application technology from Jilin University, Changchun, China, in 2007.

He is currently a Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research is focused on intelligent transportation systems, autonomous connected vehicles, swarm intelligent, and mobile computing.

Prof. Tian was awarded the Changjiang Scholars Program (Young Scholar) of Ministry of Education of China in 2017, the National Science Fund for Distinguished Young Scholars in 2018, and the Distinguished Young Investigator of China Frontiers of Engineering in 2018. He was served as the Technical Program Committee member/Chair/Co-Chair for several international conferences, including EAI 2018, ICTIS 2019, IEEE ICUS 2019, IEEE HMWC 2020, and GRAPH-HOC 2020.